

Building Tranching In-House vs. Partnering

How much effort it would take Wildcat to ship its own senior/junior tranching layer that covers the common ground of Strata, Royco Dawn and Pareto, and whether it should.

4–6 eng-months MVP BUILD EFFORT	3.5–5 months CALENDAR, 2 ENGINEERS	\$80–150k EXTERNAL AUDIT	~1–2k LOC GENUINELY NOVEL CODE
---	--	------------------------------------	--

Prepared **27 June 2026**
Ethereum mainnet

Sources: code review of v2-protocol, Strata, Royco Dawn & Pareto; on-chain reads; StrataxWildcat proposal

★ Bottom line

THE NUMBER

An in-house MVP covering the common ground of the three (two share classes over a Wildcat market, a yield waterfall with senior priority + floor and junior residual + risk premium, a junior-first loss waterfall, per-tranche NAV, a coverage ratio, async redemptions through Wildcat's withdrawal queue, plus a factory + roles) is roughly 4–6 engineer-months of build + a ~\$80–150k external audit, ≈ 3.5–5 months calendar with two strong Solidity engineers.

Reaching full feature parity with Strata/Royco (adaptive yield-split curves, depeg/valuation handling, isolated liquid-junior/illiquid-senior routing, fixed-term recovery state machine, recursive tranching/Pendle) adds another ~3–5 months, most of which is not needed for the Wintermute facility.

The novel, security-critical logic is small: ~600–2,000 lines in every one of these protocols. The cost is not line count. It is (a) getting the credit-loss waterfall right (Wildcat's risk is lumpy default risk, not smooth mark-to-market, and none of the three handle this exact case), (b) wiring redemptions through Wildcat's semi-liquid batched withdrawal queue (no reusable template exists), and (c) testing + audit to your existing bar.

The fastest path to the same outcome with zero engineering is the Strata partnership already on the table. Building in-house buys control, no revenue share, and tighter UX, at the cost of the quarter-plus above and owning the audit and risk surface permanently.

1 The three reference protocols

REPO	WHAT IT REALLY IS	TRUE SENIOR/JUNIOR ENGINE IN THE REPO?
Strata contracts–tranches	A CDO-style risk-tranching layer over any ERC4626 / yield asset. Splits one underlying into senior + junior ERC4626 tranche tokens.	Yes, fully. Three accounting engines (continuous, discrete, dynamic-yield-split), cooldown silos, isolated dual-strategy routing.
Royco Dawn royco–dawn	A from-scratch senior/junior tranching protocol. This is not the order-matching “Royco Markets”: there is no weiroll/offers/auction code at all.	Yes, the most sophisticated. Dual-NAV, impermanent-loss tracking with prioritized recovery, perpetual↔fixed-term state machine, adaptive curve, self-liquidation bonus.
Pareto USP	A synthetic dollar (USP) + a first-loss staking vault (sUSP). It buys the senior (AA) tranche of external Idle credit vaults and adds its own first-loss layer.	No, not in this repo. The actual AA/BB engine lives in Idle’s CDO contracts, not in the zip. USP only consumes it.

Implication: “what all three do” really means “what Strata and Royco do natively.” Pareto’s in-repo contributions are a different shape: a \$1-pegged senior, a staked first-loss buffer, and an epoch redemption queue with an undercollateralized pro-rata fallback.

On-chain note: the v-wmtUSDC 4626 (0xf654...) is Wildcat’s own Wildcat4626Wrapper, not a Strata contract. It wraps 0xC949... (“Wintermute Trading USD Coin”, ~\$69.65M, 8.5% APR), a different market from 0x50ebdf... (~\$2.23M), on a different borrower wallet. The tranching targets the wrapped facility.

2 The common ground (the intersection to rebuild)

Strata and Royco converge on the same eight primitives. This is the spec for an in-house “covers what they do” MVP:

- Two share classes** (senior + junior) over a single yield-bearing underlying (an ERC4626).
- Yield waterfall.** Senior gets priority + a floor/target rate; junior gets the residual; senior pays junior a risk premium (equivalently, junior funds any senior shortfall).
- Loss waterfall,** with junior as first-loss: losses cascade junior → (reserve) → senior.
- Per-tranche NAV accounting,** marked off the underlying’s value and re-synced on every interaction.
- Subordination / coverage ratio.** Caps senior leverage over junior and gates deposits/withdrawals near the floor.
- Async / semi-liquid redemption** (cooldown silos or a request/execute queue), because the underlying isn’t instantly liquid.

7 Governance: who sets the senior rate/benchmark, pauses, manages the reserve, handles shortfall/default; timelocks + guardians.

8 A factory to deploy a tranche set per underlying.

Everything beyond this list is differentiation, not common ground: Royco's fixed-term recovery state machine, IL recovery, self-liquidation, recursive tranching, Pendle; Strata's three accounting modes, daily-loss floor, depeg/valuation oracle, isolated liquidity routing, and its ~9 per-protocol strategy adapters.

③ What Wildcat already has (and where tranching attaches)

You are not starting from zero, and tranching is not a protocol fork: it's an app-layer vault stack on top of an unchanged market.

Reuse directly — this is the leg-up

- **Wildcat4626Wrapper.sol** (the deployed v-wmtUSDC) already solves the two hardest integration problems: rebasing→non-rebasing conversion via `scaleFactor`, and pass-through sanctions, with scaled-delta verification. A tranche vault is "this wrapper, but with two share classes and a waterfall." ~80% of one tranche's plumbing.
- **`scaleFactor()` as an accrual oracle.** Interest accrues monotonically, so splitting yield is straightforward. This is *easier* than the three reference protocols, which must diff a noisy ERC4626 share price.
- **Math + infra:** MathUtils (ray/bip), SafeCastLib, FIFOQueue, ReentrancyGuard, Solady ERC20/ERC4626/permit bases.
- The permissionless, arch-gated, one-per-market factory pattern (Wildcat4626WrapperFactory): copy it for a TrancheVaultFactory.
- **The sentinel integration pattern** for re-enforcing sanctions at the vault layer.
- **Engineering rigor already in place:** invariant tests, fuzzing, the a16z ERC4626 suite, external audits. A new component is expected to clear the same bar.

Architectural seam: tranching cannot live in the hooks system. Hooks are reactive/restrictive (they can revert, and only tweak APR/reserve); they cannot route funds or hold dual-class accounting. The tranche logic must be a separate vault that holds the market token (or v-wmtUSDC) and issues senior/junior shares. Hooks can assist (e.g. restrict a market so only the tranche vault may deposit), but that's optional.

④ The Wildcat-specific hard parts (where the months go)

These are the three areas with the least reusable code and the most design / risk.

- 1 CRUX Credit-loss vs mark-to-market loss.** Strata/Royco/Pareto tranche *liquid* yield where loss = a falling ERC4626 share price, detected automatically. Wildcat is undercollateralized private credit: the upside (interest) is smooth and easy to tranche, but “loss” is lumpy and discrete. It splits into delinquency (penalty APR, delayed withdrawals, principal intact) versus default or closure with a shortfall (principal impaired). You must *define* the loss trigger (When is the loan impaired? Who declares it? How is a delayed/partial recovery socialized back?) before any junior-first write-down can be coded. This is a design and likely legal question.
- 2 Redemptions through a semi-liquid, batched, pro-rata, time-delayed queue.** Wildcat withdrawals are queued into expiry-dated batches, paid as liquidity allows, with a FIFO unpaid-batch queue and pro-rata fills. A tranche vault must queue → wait for expiry → execute, then allocate the partial, delayed proceeds across senior/junior per the waterfall (senior first). None of the three handle anything this complex on exit; they assume instant or simple-cooldown ERC4626 liquidity. This is the largest piece of net-new design.
- 3 Per-user KYC / sanctions at the vault layer.** When lenders are pooled into a tranche vault, the market only sees the vault as the lender, so per-user credential and sanctions visibility is lost at the market level. The existing 4626 wrapper shows the pattern (re-check the sentinel on every entry and on share transfers); a permissioned senior/junior product re-enforces per-user gating in the vault too.

5 Effort estimate

Assumes two strong Solidity engineers, reuse of the 4626 wrapper + math libs, and your existing test/audit standard.

PHASE	SCOPE	NEW SOLIDITY (INCL. TESTS)	CALENDAR
0 — Design & risk spec	Define the credit-loss model (impairment trigger, recovery socialization), redemption allocation across batches, coupon-vs-delinquency interaction, coverage ratio + param bounds. <i>This phase drives the calendar.</i>	spec, not code	3–5 wks
1 — MVP core	2 tranche tokens; yield waterfall (senior floor + priority, junior residual + risk premium); junior first-loss; coverage-ratio gating; NAV off <code>scaleFactor</code> ; redemption orchestration through the withdrawal queue + a tranche cooldown; factory; roles; pause.	~3.5–5k LOC	6–10 wks
2 — Hardening	Invariant + fuzz tests to your bar, a16z 4626 suite, delinquency/default/partial-fill edge cases, vault-layer sanctions/KYC.	folded in	4–6 wks
3 — Audit + fixes	External audit + remediation.	—	4–8 wks (+\$80–150k)
MVP total	Common ground, audited	~4–6 eng-months	~3.5–5 months
4 — Advanced (optional)	Adaptive/dynamic yield-split curve; depeg/valuation oracle; isolated liquid-junior/illiquid-senior routing; fixed-term recovery state machine; recursive tranching / Pendle.	+3–6k LOC	+3–5 months

Why not faster, given the IP is only ~1–2k LOC? Calendar time is dominated by (1) the loss-model design, (2) the redemption integration that has no template, and (3) test + audit to a credit-protocol standard. The reference repos are 22k–48k LOC precisely because of tests/adapters/ops around a small core: ~32k of Royco's 48k is tests.

⑥ Build vs. partner

Partner (Strata, as proposed)

Tranches live in weeks, with ~zero Wildcat eng. Strata brings NAV accounting, programmable waterfalls, yield distribution, enforced mint/redeem; you bring market + joint UI. Cost: revenue share, external trust surface, less control over UX/roadmap.

Build in-house

Full control, no revenue share, native UX, reusable across every market via a factory. Cost: the quarter-plus opposite, owning the audit + ongoing maintenance, and owning the credit-loss-model risk.

Pragmatic middle

Partner to ship the Wintermute facility and learn real demand/UX, while running the cheap design phase in parallel. It de-risks the whole build; the decision on the MVP follows once the loss model and redemption design are settled.

A Reference-protocol core files (where the real IP sits)

- **Strata:** Accounting.sol (deployed continuous engine, ~628 LOC, the most direct expression of the model), DYSAccounting.sol (~1,203 LOC, advanced), StrataCD0.sol (orchestrator), Tranche.sol, strategies/spark/SparkUSDCStrategy.sol (the generic-ERC4626-wrap template, closest analog to wrapping v-wmtUSDC), base/cooldown/SharesCooldown.sol.
- **Royco Dawn:** RoycoAccountant.sol (~916 LOC: waterfalls, IL, state machine), RoycoKernel.sol (~966 LOC: coverage, self-liquidation), AdaptiveCurveYDM_V2.sol, RoycoVaultTranche.sol, Units.sol (typed NAV accounting), RoycoEntryPoint.sol (async queue).
- **Pareto:** ParetoDollarQueue.sol (~757 LOC: NAV aggregation, epoch queue, gain/loss → sUSP first-loss), ParetoDollarStaking.sol. *The AA/BB engine itself is external (Idle).*
- **Wildcat (reuse):** vault/Wildcat4626Wrapper.sol, vault/Wildcat4626WrapperFactory.sol, libraries/MarketState.sol, market/WildcatMarketWithdrawals.sol + libraries/Withdrawal.sol, libraries/MathUtils.sol.